

Security Audit

RGB (Collaborative NFT Canvas)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	31
Conclusion	32
Our Methodology	33
Disclaimers	35
About Hashlock	36

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The RGB team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

RGB is an on-chain collaborative art platform where users spend the native RGB ERC-20 (or ETH) to encode pixel data into an active canvas; once the canvas's block window closes, the contract mints a timelapse NFT that records the entire draw history and distributes its auction proceeds to contributors by pixel share, while canvases can optionally attach an RGB-denominated lottery pool whose winners are selected when the canvas is finalized.

Project Name: RGB

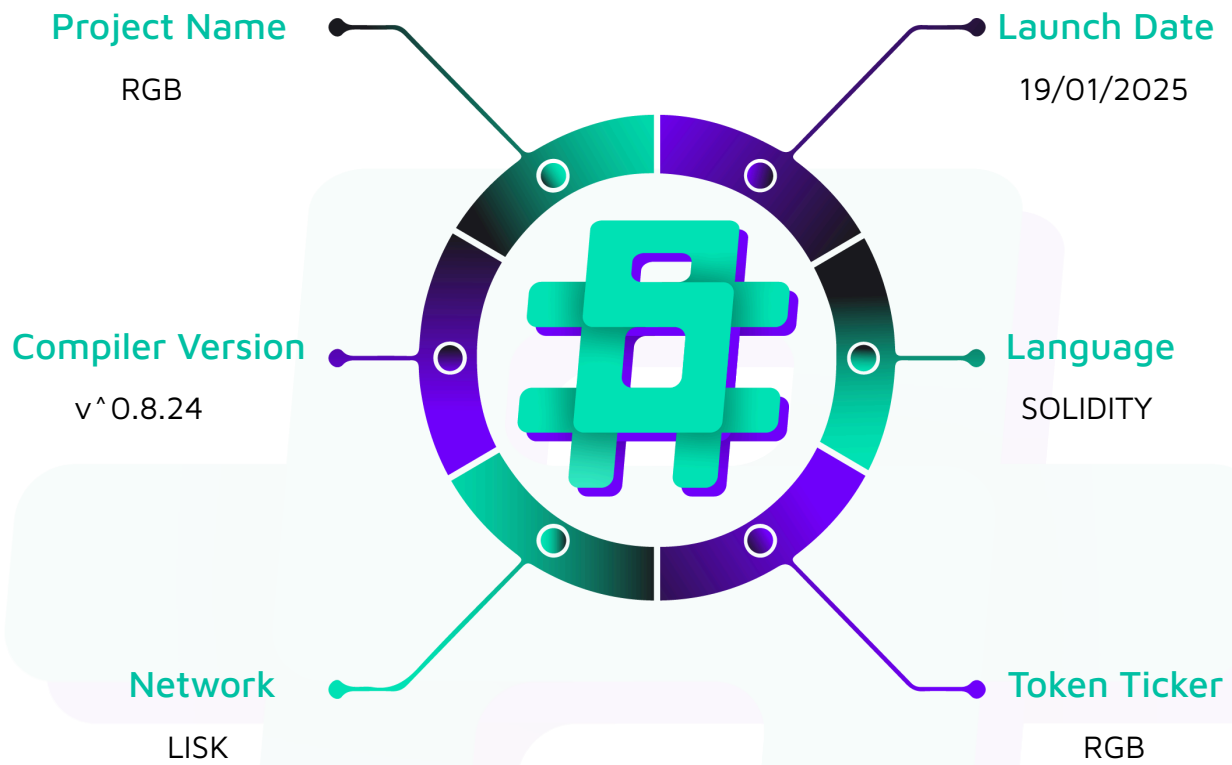
Project Type: Collaborative NFT canvas

Compiler Version: ^0.8.24

Website: <https://rgblisk.io/>

Logo:

A large, bold, blue 'RGB' logo is positioned in the lower-left quadrant of the page. The letters are thick and stylized, with the 'R' having a small circular detail at its base. The background features a large, faint, light blue and purple graphic of a hand holding a paintbrush, which is partially obscured by the 'RGB' text.

Visualised Context:

Project Visuals:

World First NFT Collaboration Platform

For Everyone!

Join us as we create history by capturing internet culture in **NFT** format using the **RGB** platform powered by Lisk.

[Buy](#) [Import Token](#)

The visual features a light purple background with a large, faint hand silhouette. On the right, there's an illustration of an easel with a canvas showing a yellow triangle and a pink circle. Above the easel are icons: a blue paperclip, a green circular arrow, a blue Wi-Fi symbol, and a red heart. A small framed picture with a pink circle and a yellow square is also visible.



Audit Scope

We at Hashlock audited the solidity code within the RGB project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	RGB Smart Contracts
Platform	Ethereum / Solidity
Audit Date	June, 2025
Contract	RgbToken.sol
Contract MD5 Hash	9496ee93bb2caf3369ae7e629c36e182
Audited Contract Address	0x0bf4a86BA09fb1BF45b10D49F10953F6bc8E8A3c
Audited GitHub Commit Hash	bc94ced3477cc05a5dfeba4ba4cc08421028583f
Fix Review Contract Address	0xBa655C53cF01b461C94e2C3cdbD678A5D46C4ef8
Fix Review GitHub Commit Hash	f8de8d4e283a30dca246ef55d5a1a5e2b1677edf

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. The protocol uses OpenZeppelin's ERC20 implementation but has custom implementations for other standard contracts. Notably, the contract uses its own versions of Ownable and SafeMath libraries rather than utilizing the full OpenZeppelin suite.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 Medium severity vulnerabilities

1 Low severity vulnerabilities

1 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
RgbToken.sol - Allows users to: - Draw pixels on available canvases - Place stamps on available canvases - Participate in lottery systems associated with canvases - Pay for drawing transactions in ETH or RGB tokens - Claim lottery winnings (through push payments) - Allows canvas owners (contract owner) to: - Create new canvases with customizable properties - Change canvas settings before they become active - Remove canvases that haven't started yet - Complete canvases after their end block - Set drawing tax percentages - Set lottery tax percentages - Register external lottery and stamp contracts	Contract achieves this functionality but with some limitations.

Code Quality

This audit scope involves the smart contracts of the RGB project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the RGB project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

The protocol uses OpenZeppelin's ERC20 implementation but has custom implementations for other standard contracts. Notably, the contract uses its own versions of Ownable and SafeMath libraries rather than utilizing the full OpenZeppelin suite.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] RgbToken#_validateStamps - Coordinate-based validation in the stamp function allows bypassing pixel transaction limits

Description

The RgbToken contract enforces a `maxPixelsPerTransaction` limit to control resource usage, but this limit is incorrectly applied in the `stamp` function. While coordinate count is properly validated, the contract fails to verify that the total pixel count from stamps respects this limit, allowing users to affect significantly more pixels than intended.

Vulnerability Details

The RGB token contract implements two methods for drawing on a canvas: direct pixel drawing via the `draw` function and stamp placement via the `stamp` function. Both functions should be subject to the same `maxPixelsPerTransaction` limit.

The validation chain in the `stamp` function has a significant gap:

1. First, coordinates are validated in `_validateCoords`:

```
function _validateCoords(bytes calldata coords, IRgbCanvas.Canvas storage canvas) private
view returns (uint256) {

    // ...

    uint256 numOfCoords = coords.length / canvas.coordLength;

    // ...

    if (numOfCoords > canvas.drawSettings.maxPixelsPerTransaction) {

        revert TooManyCoords(coords);

    }
}
```

```
// ...
}
```

2. Then stamps are validated to match coordinates in `_validateStamps`:

```
function _validateStamps(uint64[] calldata stamps, uint256 numOfCoords) private view
returns (uint256) {

    // ...

    if (stamps.length != numOfCoords) {

        revert InvalidStamps(stamps);

    }

    uint256 numOfPixels = 0;

    for (uint256 i = 0; i < stamps.length; ++i) {

        numOfPixels = numOfPixels.add(_rgbStamps.getPixelCost(stamps[i]));

    }

    // ...

}
```

The key issue is how `numOfPixels` is calculated: by summing `_rgbStamps.getPixelCost(stamps[i])` for each stamp ID. This means each stamp can represent multiple pixels, with the count determined by the external contract's `getPixelCost` function.

The complete validation chain is:

- `_validateCoords` ensures `numOfCoords <= maxPixelsPerTransaction`
- `_validateStamps` ensures `stamps.length == numOfCoords`
- But nothing ensures the total number of pixels (summed from `getPixelCost`) `<= maxPixelsPerTransaction`

While each coordinate places exactly one stamp, each stamp can represent multiple pixels. The limitation is enforced at the coordinate/stamp count level, not at the final pixel impact level.



Impact

The validation gap in the `stamp` function circumvents a core contract constraint by allowing users to affect significantly more pixels than the `maxPixelsPerTransaction` limit intended.

Recommendation

Add a secondary validation check in the `_validateStamps` function to ensure the total pixel count (not just coordinate count) does not exceed the maximum allowed per transaction:

```
function _validateStamps(
    uint64[] calldata stamps,
    uint256 numOfCoords,
+   IRgbCanvas.Canvas storage canvas
) private view returns (uint256) {
    _validateStampsContract();

    if (stamps.length != numOfCoords) {
        revert InvalidStamps(stamps);
    }

    uint256 numOfPixels = 0;
    for (uint256 i = 0; i < stamps.length; ++i) {
        numOfPixels = numOfPixels.add(_rgbStamps.getPixelCost(stamps[i]));
    }

+   if (numOfPixels > canvas.drawSettings.maxPixelsPerTransaction) {
```

```

+     revert TooManyPixels(numOfPixels, canvas.drawSettings.maxPixelsPerTransaction);
+ }

    return numOfPixels;
}

```

Status

Acknowledged

[M-02] RgbToken#_chargePixels - Hardcoded gas formula in pixel fee calculation leads to economic imbalance

Description

The RgbToken contract uses a hardcoded gas formula to calculate transaction fees instead of using actual gas measurements. This implementation creates a discrepancy between the charged fees and actual gas costs, resulting in either users overpaying or the protocol subsidizing transactions.

Vulnerability Details

In the RgbToken::_chargePixels function, a fixed formula calculates the gas consumption for drawing pixels on a canvas:

```

// 128807 base gas + 698 gas per pixel (coords length = 3 x num of pixels)
uint256 requiredEth = _chargeGasTaxFee(
    (numOfPixels.mul(698).add(128807)),
    _drawTax,
    owner()
);

```

This formula assumes a static cost model where:



- Base cost is exactly 128,807 gas units
- Each pixel costs exactly 698 gas units

However, actual gas costs are dynamic and depend on:

1. Network conditions and gas pricing
2. EVM optimizations in the current Solidity compiler
3. Contract state at execution time
4. Canvas configuration (coordinate size, color encoding)

The `_chargeGasTaxFee` function applies the specified tax percentage to this estimated gas amount and charges users accordingly:

```
function _chargeGasTaxFee(  
    uint256 gasAmount,  
    uint16 taxPercentage,  
    address recipient  
) private returns (uint256) {  
    uint256 gasCost = gasAmount.mul(tx.gasprice);  
    uint256 tax = _calculateTax(gasCost, taxPercentage);  
  
    if (tax > 0) {  
        payable(recipient).transfer(tax);  
    }  
  
    return tax;  
}
```

This implementation produces an economic imbalance that grows as the actual gas consumption diverges from the hardcoded formula over time.

Proof of Concept

Create a `/test` folder with a `RGBTokenGasTest.t.sol` file in it, paste the following PoC, and run it:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "forge-std/Test.sol";
import "../contracts/RgbToken.sol";

contract RgbTokenGasFormulaTest is Test {
    receive() external payable {}

    RgbToken private rgbToken;
    address private user;
    uint32 private constant CANVAS_ID = 1;

    function _coords(
        uint256 n,
        uint32 len
    ) internal pure returns (bytes memory b) {
        b = new bytes(n * len);
        for (uint256 i; i < n; ++i) b[i * len] = bytes1(uint8(i));
    }

    function setUp() public {
        user = makeAddr("user");

        rgbToken = new RgbToken(); // this contract is protocol owner
        rgbToken.changeDrawTax(1_000); // 10 % (bps)

        /* 32x32 canvas, zero per-pixel cost */
        IRgbCanvas.CanvasProperties memory props = IRgbCanvas.CanvasProperties({
            length: 1024,
            dimensions: hex"2020",
            encoding: 2,
            colourPalette: new bytes(12) // 4 colours x RGB
        });
    }
}
```

```

IRgbCanvas.DrawSettings memory draw = IRgbCanvas.DrawSettings({
    maxPixelsPerTransaction: 1024,
    timeBetweenDraws: 1,
    costPerPixel: IRgbCanvas.Cost({
        currency: IRgbCanvas.Currency.ETH,
        value: 0
    })
});

IRgbCanvas.BlockRange memory br = IRgbCanvas.BlockRange({
    start: block.number + 2,
    end: block.number + 1000
});

IRgbCanvas.LotterySettings memory lot; // default-zero structs

rgbToken.createCanvas(CANVAS_ID, props, draw, br, lot, "gas-test");
vm.roll(block.number + 3); // canvas live
vm.deal(user, 10 ether); // fund test user
}

function testGasFormulaMismatch() public {
    vm.txGasPrice(1 gwei);

    uint256 pixels = 1000; // sample
    bytes memory cds = _coords(pixels, 2); // coordLen = 2
    bytes memory cls = new bytes((pixels * 2 + 7) >> 3); // 2-bit colours

    uint256 ownerBalBefore = address(this).balance;
    uint256 gasBefore = gasleft();

    vm.prank(user);
    rgbToken.draw{value: 5 ether}(CANVAS_ID, cds, cls);

    uint256 gasUsed = gasBefore - gasleft(); // real gas inside draw()
    uint256 realFee = address(this).balance - ownerBalBefore;

    uint256 shouldPay = (gasUsed * 1 gwei * 1_000) / 10_000; // 10 % of real gas

```

```

uint256 diff = realFee > shouldPay
    ? realFee - shouldPay
    : shouldPay - realFee;

// require a >5 % mismatch to prove the formula is wrong
assertGt(diff, shouldPay / 20, "hard-coded gas formula still accurate");
}
}

```

Impact

This vulnerability creates economic imbalances in two possible directions:

1. If actual gas costs exceed the formula's prediction:
 - Contract owner subsidizes user transactions
 - Protocol becomes economically unsustainable over time
 - Risk of protocol insolvency in high-gas market conditions
2. If actual gas costs are less than the formula's prediction:
 - Users overpay for transactions
 - Creates poor user experience
 - Potential competitive disadvantage against more efficient protocols

Recommendation

Compute actual gas usage rather than using a hard-coded formula. This approach automatically tracks compiler and EVM changes without requiring manual updates:

```

function _chargePixels(uint256 numOfPixels, Canvas storage canvas) private returns
(uint32) {
    uint32 participantId = _getParticipantId(canvas.canvasId, msg.sender);
    _validateTimeBetweenDraws(participantId, canvas);

+   uint256 startGas = gasleft();

-   // 128807 base gas + 698 gas per pixel (coords length = 3 x num of pixels)
-   uint256 requiredEth = _chargeGasTaxFee(
-       (numOfPixels.mul(698).add(128807)),
-       _drawTax,
-       owner()

```

```

- );

// Remaining logic for charging draw cost and lottery cost
uint256 drawCost = _chargeDrawCost(numOfPixels, canvas);
uint256 lotteryCost = 0;

if (canvas.lotterySettings.isLottery) {
    lotteryCost = _chargeLotteryCost(numOfPixels, canvas);
    // Update lottery entries
    uint256 totalEntries = _participants[canvas.canvasId]
        .participants[participantId]
        .numOfEntries
        .add(numOfPixels);
    _participants[canvas.canvasId]
        .participants[participantId]
        .numOfEntries = totalEntries;
}

+ // Calculate gas used and charge tax on actual consumption
+ uint256 gasUsed = startGas - gasleft();
+ uint256 gasTax = _chargeGasTaxFee(gasUsed, _drawTax, owner());
+ uint256 requiredEth = gasTax + drawCost + lotteryCost;

// Checking ethereum payed is enough
if (requiredEth > msg.value) {
    revert InsufficientPayment(msg.value, requiredEth);
}

uint256 remainder = msg.value.sub(requiredEth);

// Pay back remainder
if (remainder > 0) {
    payable(msg.sender).transfer(remainder);
}

_participants[canvas.canvasId].lastDrawnHeight[participantId] = block.number;
return participantId;
}

```

Status

Resolved

[M-03] RgbToken#_chargePixels/_transferFromPayee/_withdrawContract - Use of `transfer()` for ETH payments causes risks for contract recipients

Description

The `RgbToken` contract uses Solidity's `.transfer()` method to send ETH in multiple locations, which forwards a fixed amount of 2300 gas. This fixed gas stipend can cause transactions to fail when the recipient is a contract with a complex receive function, potentially blocking critical functionality like refunds and lottery payouts.

Vulnerability Details

The `.transfer()` method is used in multiple locations throughout the `RgbToken` and `Taxable` contracts to send ETH to various recipients:

1. In `RgbToken::_chargePixels()` for returning excess ETH to users:

```
if (remainder > 0) {  
    payable(msg.sender).transfer(remainder);  
}
```

2. In `Taxable::_chargeGasTaxFee()` for sending tax fees:

```
payable(owner).transfer(minFee);
```

3. In `RgbToken::_transferFromPayee()` for ETH transfers:

```
if (currency == Currency.ETH) {  
    payable(to).transfer(value);  
    return value;  
}
```

4. In `RgbToken::_withdrawContract()` for ETH transfers including lottery payouts:

```
if (currency == Currency.ETH) {  
    payable(to).transfer(value);  
}
```

Using `.transfer()` introduces two specific risks:

1. Gas costs of EVM instructions may change significantly during hard forks which may previously assume fixed gas costs. EIP 1884 as an example, broke several existing smart contracts due to a cost increase of the SLOAD instruction.
 2. If the recipient is a contract or a multisig safe, with a `receive/fallback` function which requires `>2300` gas, e.g safes that execute extra logic in the `receive/fallback` function, the transfer function will always fail for them due to out of gas errors.
- This issue is particularly problematic in `completeCanvas` where lottery winnings are distributed to participants. If any winner uses a smart contract wallet, the entire canvas completion process could be blocked.

Impact

This vulnerability could prevent:

- Users from receiving refunds of excess ETH
- The protocol owner from receiving tax fees if using a contract wallet
- Lottery winners from receiving their prizes if using smart contract wallets
- Canvas completion if any lottery winner uses a smart contract wallet

Recommendation

Replace all instances of `.transfer()` with `.call()` for ETH transfers while adding a non reentrant modifier.

Status

Resolved

[M-04] **RgbToken#_getParticipantId/completeCanvas** - Unbounded participant and winner arrays in RGB lottery system enable permanent freezing of funds

Description

The RgbToken lottery implementation fails to set practical limits on both participant and winner array sizes. This dual-vector vulnerability can result in gas limit exhaustion at two points: when encoding the participant array for the lottery draw and when processing winner payouts. Either condition permanently prevents canvas completion and locks lottery funds in the contract.

Vulnerability Details

The vulnerability manifests in two sequential stages of the lottery process:

Stage 1: Unbounded Participant Array

The `_getParticipantId` function is the entry point for registering lottery participants and contains only a theoretical limit check:

```
function _getParticipantId(uint32 canvasId, address participant) private returns (uint32)
{
    CanvasParticipants storage participants = _participants[canvasId];

    if (participants.participants.length == 0 ||

participants.participantIdToAddress[participants.addressToParticipantId[participant]] !=
participant) {

        if (participants.participants.length == type(uint32).max) {
            revert MaxParticipants();
        }

        uint32 participantId = participants.participants.length.toUint32();
```



```

        participants.participants.push(IRgbLottery.Participant(participantId, 0));

        participants.addressToParticipantId[participant] = participantId;

        participants.participantIdToAddress[participantId] = participant;

        emit ParticipantAdded(canvasId, participant, participantId);

    }

    return participants.addressToParticipantId[participant];
}

```

The check `if (participants.participants.length == type(uint32).max)` allows up to 4,294,967,295 participants, which is effectively no practical limit. When the entire participant array must be passed to the lottery contract in `completeCanvas`:

```

IRgbLottery.Winner[] memory winners = _rgbLottery.drawLottery(

    _participants[canvasId].participants, // <-- Entire array passed

    canvas.lotterySettings.poolAllocations,

    canvas.lotterySettings.allocationChances,

    canvas.lotteryPool

);

```

The ABI encoding of this participants array requires copying every element to memory. Even for far fewer than the maximum (e.g., 1 million participants), this would require approximately 64MB of memory and far exceed the block gas limit of ~30M gas on Ethereum mainnet.

Stage 2: Unbounded Winner Array

If the participant array encoding succeeds, the function then processes all lottery winners in a single transaction without limiting their number:

```

for (uint32 i = 0; i < winners.length; ++i) {

    _withdrawContract(

        _participants[canvasId].participantIdToAddress[winners[i].participantId],

```

```

        canvas.lotterySettings.costPerPixel.currency,

        winners[i].amount

    );

    canvas.lotteryPool = canvas.lotteryPool.sub(winners[i].amount);
}

```

Each iteration of this loop:

- Performs an external value transfer via `_withdrawContract`
- Updates contract state by reducing `canvas.lotteryPool`
- Consumes approximately 5,000+ gas per winner

Even if only a fraction of participants are chosen as winners, this can still exceed block gas limits with a few thousand winners.

Impact

While this vulnerability is low likelihood, it has severe consequences:

1. Lottery funds (ETH or RGB tokens) become permanently frozen in the contract
2. Winners can never receive their prizes
3. The canvas remains in an incomplete state indefinitely
4. Canvas owners cannot withdraw remainder amounts or publish final artwork
5. A malicious actor could target high-value canvases with substantial lottery pools
6. Layer-2 deployments with cheaper gas costs would make this attack economically feasible

Recommendation

Implement a three-part solution to address both attack vectors:

1. Add a reasonable participant limit per canvas.
2. Switch to a pull-based payment system for lottery winnings.
3. For extra safety, implement pagination for processing large arrays.

Status

Resolved

Low

[L-01] RgbToken#RegisterLotteryContract/registerStampsContract - Missing zero-address validation in contract registration functions allows temporary loss of functionality

Description

The RgbToken contract implements two registration functions, `registerLotteryContract` and `registerStampsContract`, which allow the owner to set the addresses of critical external contracts. However, these functions lack zero-address validation checks, permitting the owner to accidentally set either contract to `address(0)`. While the contract does validate these addresses at the time of use through the `_validateLotteryContract` and `_validateStampsContract` functions, allowing invalid state to be set in the first place is poor practice and can lead to temporary service disruption if the owner makes a mistake.

Recommendation

Add zero-address validation in both registration functions:

```
+ error InvalidAddress();

function registerLotteryContract(
    address rgbLotteryAddress
) external onlyOwner {
+   if (rgbLotteryAddress == address(0)) revert InvalidAddress();
    _rgbLottery = IRgbLottery(rgbLotteryAddress);
    emit LotteryContractRegistered(rgbLotteryAddress);
}

function registerStampsContract(
```

```
    address rgbStampsAddress

    ) external onlyOwner {

+   if (rgbStampsAddress == address(0)) revert InvalidAddress();

        _rgbStamps = IRgbStamps(rgbStampsAddress);

        emit StampsContractRegistered(rgbStampsAddress);

    }
```

Status

Resolved

QA

[Q-01] RgbToken - Unused participantIdToIndex mapping in CanvasParticipants struct wastes storage

Description

The RgbToken contract defines a CanvasParticipants struct that includes a mapping called participantIdToIndex. This mapping is declared but never used throughout the entire contract, meaning it consumes storage space and adds unnecessary complexity without providing any functionality. While the other mappings in this struct (participantIdToAddress, addressToParticipantId, and lastDrawnHeight) are actively used for tracking participants and enforcing draw rate limits, the participantIdToIndex mapping serves no purpose.

Recommendation

Remove the unused mapping from the struct definition.

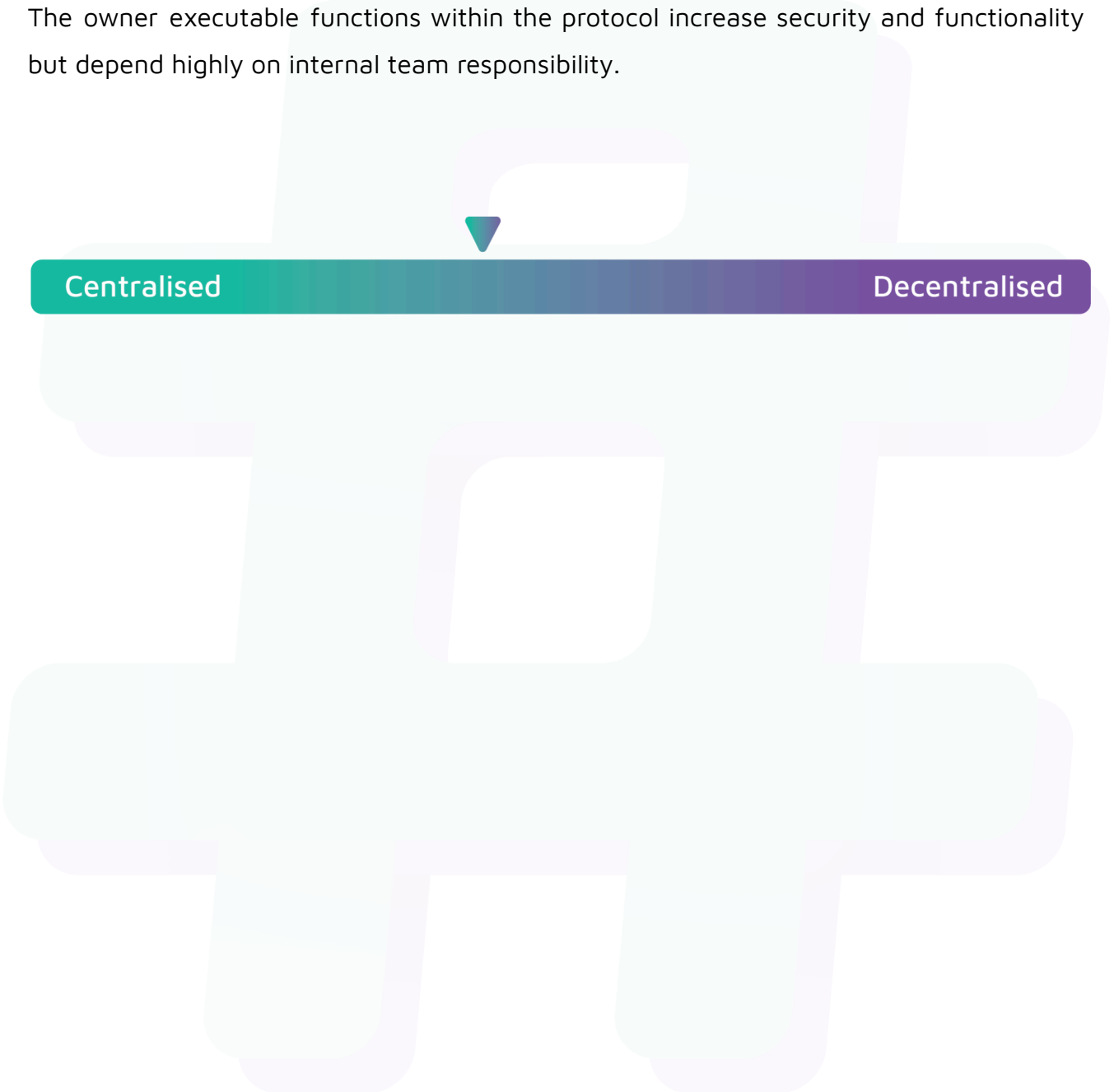
Status

Resolved

Centralisation

The RGB project values controlled ecosystem management and operational flexibility over full decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the RGB project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.